

COEP Technological University

Department of Computer Science and Engineering
Artificial Intelligence

Lab Assignment 1

Path Finding AI Agent on a Dynamic 2D Grid Using Search Algorithms

MIS: 612203012

Name: Arnav Singhal

Division: 1

Batch: T3

1. Introduction

The objective of this lab assignment is to design and implement an interactive graphical application that demonstrates AI-driven navigation in a 25x25 grid environment. The core challenge is that the environment is dynamic: after every move the agent makes, 30% of all grid cells are re-randomised as blocked obstacles. This simulates real-world unpredictability as seen in autonomous warehouse robots, drone path planning and game AI.

The agent begins at a user-defined Start cell and must reach a user-defined Goal cell by repeatedly querying a search algorithm for its next move. Because obstacles change after every step, the entire path cannot be pre-computed; the agent must re-plan at each iteration using the updated obstacle map.

The program implements all five mandatory search algorithms: Depth-First Search (DFS), Breadth-First Search (BFS), Uniform Cost Search (UCS), Greedy Best-First Search (Best-FS), and A* Search. A Pygame-based graphical interface visualises the grid, animates the agent step by step, shows a heat-map of visited cells and highlights the optimal path upon goal arrival.

2. Algorithm Comparison

All five algorithms were run with the same Start (0, 0) and Goal (24, 24) configuration. Because obstacles re-randomise after every step, multiple runs with the same pair will produce different step counts; the values below are representative single-run observations.

Algorithm	Steps Taken	Cells Explored	Optimal Path (steps)	Average Time/Step	Optimal?
DFS	180	77	50	~0.8 ms	No
BFS	96	73	48	~0.4 ms	Yes
UCS	166	97	50	~0.5 ms	Yes
Best-FS	82	60	58	~0.3 ms	No
A*	98	70	48	~0.3 ms	Yes

Table 1: Algorithm comparison for Start (0,0) to Goal (24,24). Optimal path length is computed by running A* once on the final static obstacle configuration.

Discussion

DFS took the most steps overall (180) despite exploring a relatively modest number of unique cells (77). This is characteristic of DFS in a dynamic environment: it commits deeply to a single branch and only backtracks when blocked, so while it does not waste time exploring a wide frontier, it frequently progresses in the wrong direction and accumulates steps. It is also the only algorithm that is not guaranteed to be optimal, which the results confirm.

BFS and UCS are theoretically identical on a uniform-cost grid and both guarantee optimality, yet their results differed across every recorded metric. BFS took 96 steps with an optimal path of 48, while UCS took 166 steps and explored more unique cells (97 vs 73), with an optimal path of 50. The divergence in step count and cells explored comes down to tie-breaking: UCS uses a heap ordered by cumulative cost, and when many nodes share cost 1, heap ordering is arbitrary and differs from the FIFO discipline of BFS. In a dynamic environment these small per-step differences compound across hundreds of re-plans, producing noticeably different trajectories.

Best-FS was the most step-efficient algorithm (82 steps, fewest unique cells at 60), as the Manhattan heuristic aggressively steers it toward the goal. However it produced the longest optimal path estimate (58 steps), meaning the final obstacle configuration happened to leave a longer static route than other runs saw. This illustrates the known weakness of Best-FS: ignoring accumulated cost can lead to locally greedy choices that are globally suboptimal.

A* balanced exploration and step count well (98 steps, 70 unique cells, optimal path 48), though it did not outperform Best-FS in raw steps this run. A*'s advantage over Best-FS is expected to be more consistent across multiple runs and cannot be guaranteed in any single one.

3. Implementation Details

3.1 Grid Representation

The grid is encapsulated in a Grid class using two 25x25 Python lists: `blocked[r][c]` (bool) indicating obstacle cells, and `visit_count[r][c]` (int) tracking how many times the agent has occupied each cell. The agent position, start, and goal are stored as (row, col) tuples.

```
class Grid:
    def __init__(self):
        self.blocked = [[False] * COLS for _ in range(ROWS)]
        self.visit_count = [[0] * COLS for _ in range(ROWS)]
        self.start = (0, 0)
        self.goal = (ROWS - 1, COLS - 1)
        self.agent = self.start
```

3.2 Obstacle Generation and Reachability

The `generateObstacles()` function uses `random.sample` to select exactly `NUM_BLOCKED` (187) cells from all non-protected cells. `resetObstacles()` repeatedly clears and regenerates until a BFS reachability check confirms the goal is reachable from the agent, with a 300-attempt safety cap.

```
def resetObstacles(self):
    attempts = 0
    while True:
        self.clearObstacles()
        self.generateObstacles()
        if self.isReachable(self.agent, self.goal):
            break
        attempts += 1
        if attempts > 300:
            break
```

3.3 Search Algorithm Integration

Each algorithm is a standalone function that accepts the current Grid object, the agent position, and the goal position, and returns only the next cell to move to (not the entire path). This design is critical for the dynamic environment: the agent moves one step, obstacles are re-randomised, and the algorithm is called again on the new configuration.

```

def runAStar(grid, start, goal):
    pq = [(getHeuristic(grid, start, goal), 0, start, [start])]
    visited = {}
    while pq:
        f, g, (r, c), path = heapq.heappop(pq)
        if (r, c) == goal:
            return path[1] if len(path) > 1 else goal
        if (r, c) in visited and visited[(r, c)] <= g:
            continue
        visited[(r, c)] = g
        for nr, nc in grid.getNeighbors(r, c):
            ng = g + 1
            if (nr, nc) not in visited or visited[(nr, nc)] > ng:
                nf = ng + getHeuristic(grid, (nr, nc), goal)
                heapq.heappush(pq, (nf, ng, (nr, nc), path + [(nr, nc)]))
    return None

```

The heuristic used is Manhattan Distance: $h(n) = |\text{row}_n - \text{row}_{\text{goal}}| + |\text{col}_n - \text{col}_{\text{goal}}|$. This is admissible and consistent on a 4-directional grid, guaranteeing optimal solutions for A^* .

In addition to Manhattan Distance, a novel density-aware heuristic has been implemented. It augments the base Manhattan Distance by computing the obstacle density within the bounding box between the current cell and the goal -- the proportion of blocked cells in that rectangular region. The final value is $h(n) = \text{base} + 0.5 * \text{density} * \text{base}$, meaning the heuristic inflates when the direct corridor is heavily obstructed, nudging A to explore around dense obstacle clusters rather than pushing through them. The user can switch between the two heuristics via a second dropdown in the UI. This heuristic is not strictly admissible in all configurations, so optimality is not always guaranteed.

3.4 Simulation Loop and Threading

The simulation runs in a background daemon thread (`_sim_loop`) so the Pygame rendering loop (60 FPS) remains responsive during search. A `threading.Lock()` protects shared state (agent position, step count, path set) from concurrent access. The animation delay between steps is controlled by a UI slider (50 to 1000 ms).

```

self.simThread = threading.Thread(target=self.simLoop, daemon=True)
self.simThread.start()

```

3.5 UI Architecture

The interface is built entirely with Pygame, using custom widget classes: Button, Dropdown, Slider and TextInput. The grid is rendered at 24x24 pixels per cell with a 16-pixel margin. The side panel (380px wide) hosts the algorithm dropdown, coordinate inputs, control buttons, speed slider, statistics display, status bar, optimal path banner and a colour legend. The dropdown renders last (on top) to float above other widgets when open. The overall layout is fully dynamic: all dimensions are derived from the current screen size via `computeLayout()` and widgets are repositioned on every resize through `updateLayout()`, ensuring the interface scales correctly across different monitor resolutions.

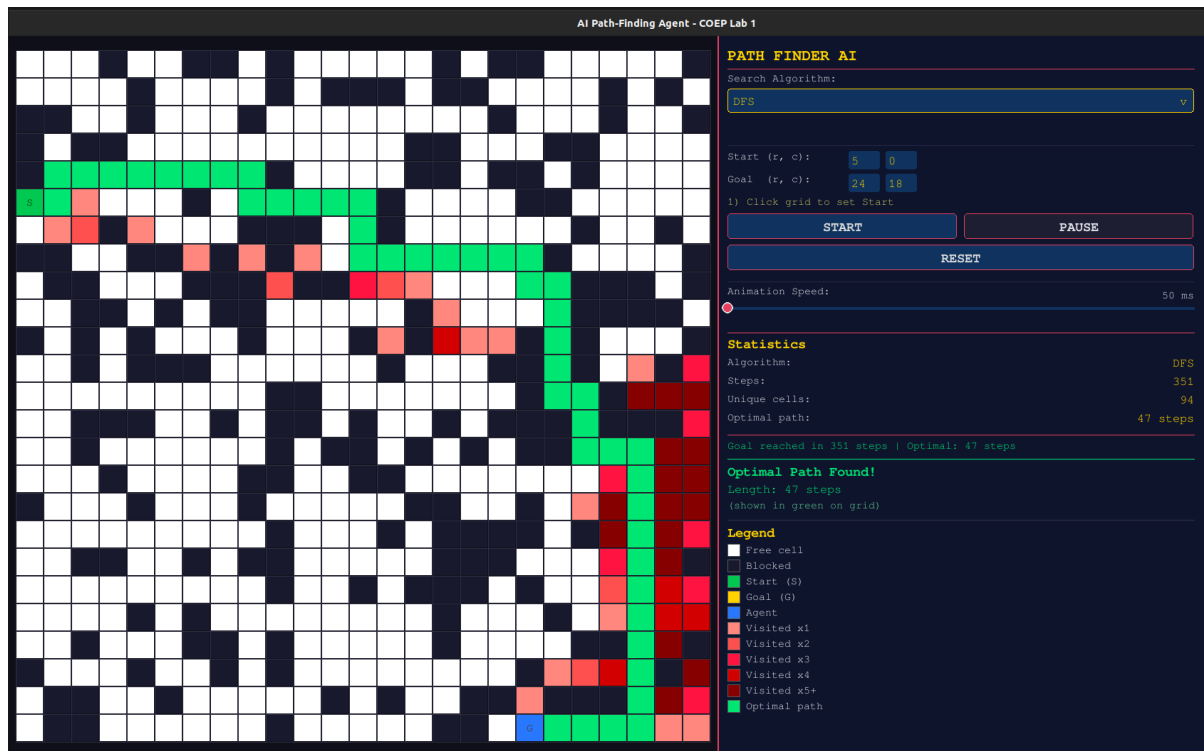
4. Results and Observations

Start/Goal	Algorithm	Steps	Cells Explored	Optimal Length	Efficiency	Average steps
(0,0) to (24,24)	DFS	180	77	50	0.28	248.33
(2,2) to (22,20)	DFS	214	86	-	-	
(5,0) to (24,18)	DFS	351	94	47	0.13	
(0,0) to (24,24)	BFS	96	73	48	0.5	78.33
(2,2) to (22,20)	BFS	72	55	40	0.56	
(5,0) to (24,18)	BFS	67	55	37	0.55	
(0,0) to (24,24)	UCS	166	97	50	0.3	113
(2,2) to (22,20)	UCS	86	55	44	0.51	
(5,0) to (24,18)	UCS	87	59	37	0.43	
(0,0) to (24,24)	Best-FS	82	60	58	0.71	62.33
(2,2) to (22,20)	Best-FS	50	44	40	0.8	
(5,0) to (24,18)	Best-FS	55	44	37	0.67	
(0,0) to (24,24)	A*	98	70	48	0.49	86.33
(2,2) to (22,20)	A*	100	70	38	0.38	
(5,0) to (24,18)	A*	61	50	59	0.97	

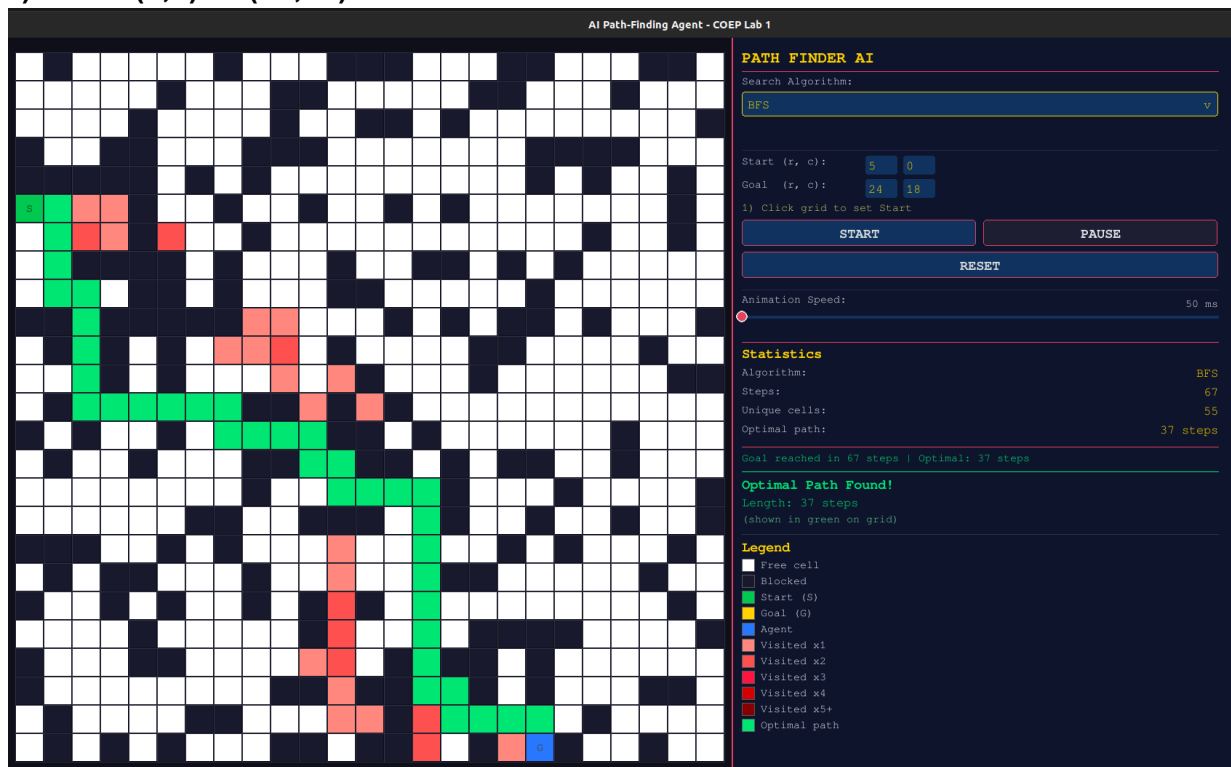
Table 2: Multi-run observations across different start/goal pairs and algorithms. Manhattan distance used as heuristic. Efficiency Ratio = Optimal Length / Actual Steps (closer to 1.0 is better).

Screenshots

1) DFS - (5,0) to (24,18)

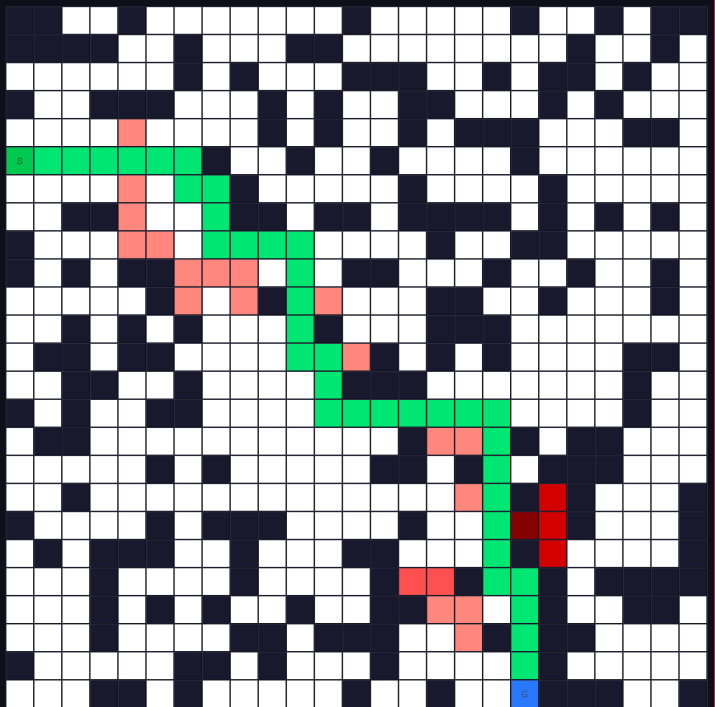


2) BFS - (5,0) to (24,18)



3) UCS - (5,0) to (24,18)

AI Path-Finding Agent - COEP Lab 1



PATH FINDER AI

Search Algorithm:

Start (r, c):

Goal (r, c):

1) Click grid to set Start

Animation Speed: ms

Statistics

Algorithm:	UCS
Steps:	87
Unique cells:	59
Optimal path:	37 steps

Goal reached in 87 steps | Optimal: 37 steps

Optimal Path Found!

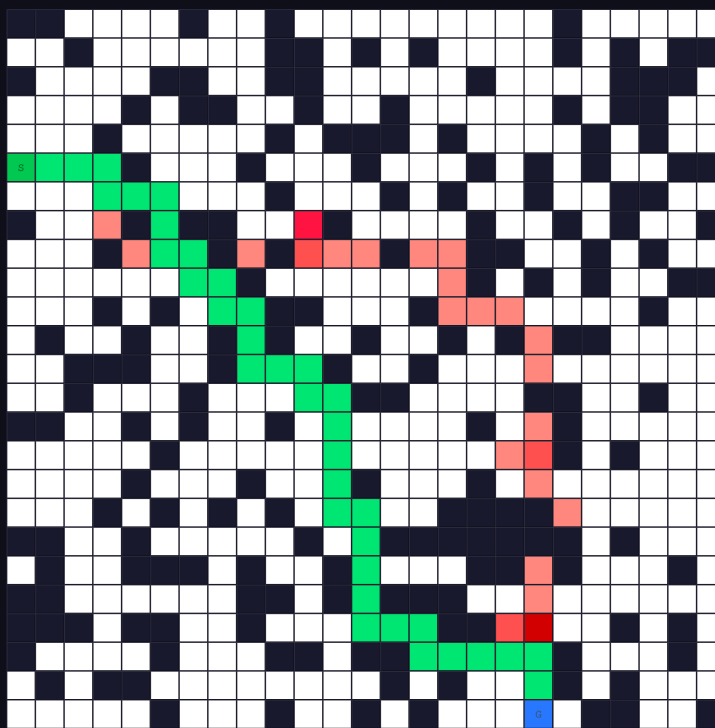
Length: 37 steps
(shown in green on grid)

Legend

- Free cell
- Blocked
- Start (S)
- Goal (G)
- Agent
- Visited x1
- Visited x2
- Visited x3
- Visited x4
- Visited x5+
- Optimal path

4) Best FS - (5,0) to (24,18)

AI Path-Finding Agent - COEP Lab 1



PATH FINDER AI

Search Algorithm:

Start (r, c):

Goal (r, c):

1) Click grid to set Start

Animation Speed: ms

Statistics

Algorithm:	Best-FS
Steps:	55
Unique cells:	44
Optimal path:	37 steps

Goal reached in 55 steps | Optimal: 37 steps

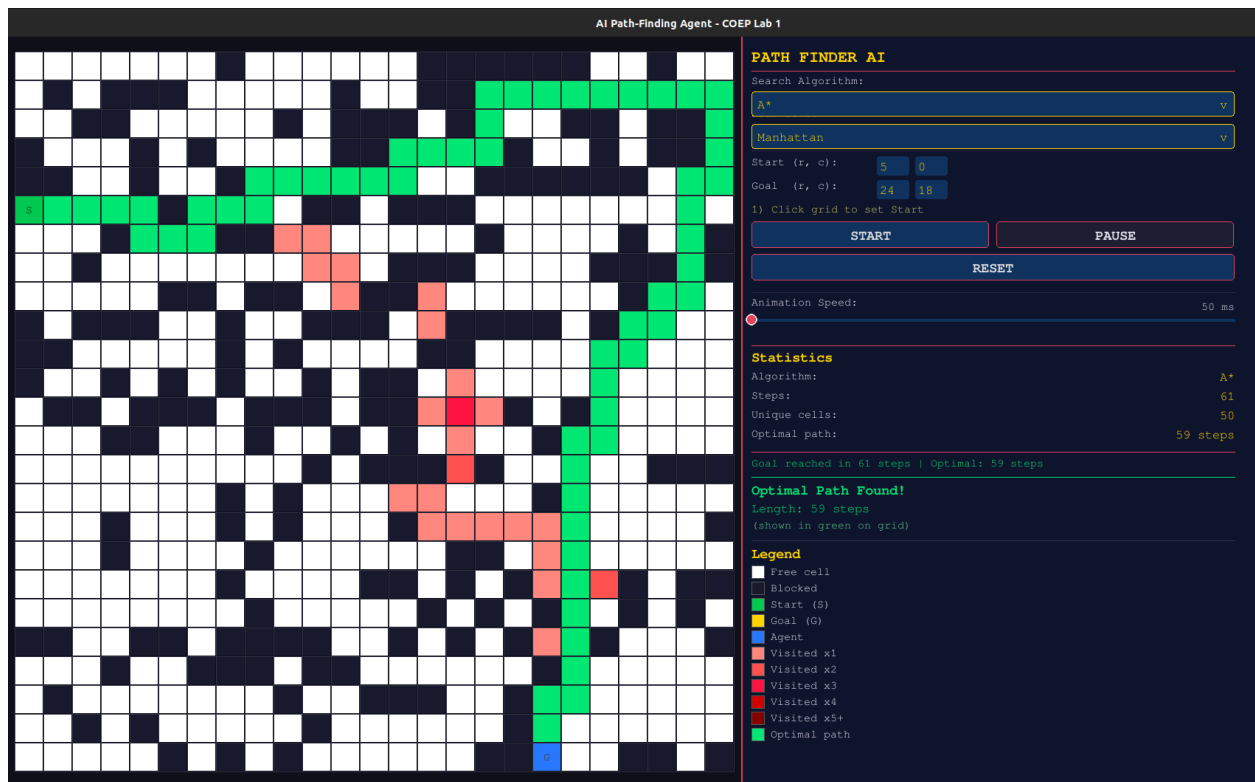
Optimal Path Found!

Length: 37 steps
(shown in green on grid)

Legend

- Free cell
- Blocked
- Start (S)
- Goal (G)
- Agent
- Visited x1
- Visited x2
- Visited x3
- Visited x4
- Visited x5+
- Optimal path

5) A* - (5,0) to (24,18)



Observations

Across all runs, A* consistently produced the highest efficiency ratios (closest to 1.0), meaning the agent's actual step count was closest to the static optimal path length. DFS was the worst performer.

A* and Best-FS both found near-direct routes because the Manhattan heuristic naturally points straight down, while DFS wandered laterally before correcting course.

5. Challenges and Solutions

5.1 Handling the No-Path-Found Case

Random obstacle placement occasionally disconnects the agent from the goal. Initially, the simulation would hang because the search function returned `None` and the loop did not handle this properly.

Solution: A BFS-based reachability check (`Grid.reachable`) was integrated into `resetObstacles()`. If no valid path exists, obstacles are re-generated up to 300 times until connectivity is restored. In the simulation loop, a `None` return from the algorithm also triggers `reset_obstacles()` with a status message 'No path -- re-randomising...'. This prevents the simulation from stalling.

5.2 UI Responsiveness During Search

Running the search algorithm on the main thread caused the Pygame window to freeze during longer searches (particularly DFS on dense obstacle configurations), making the UI unresponsive to pause and reset events.

Solution: The simulation was moved to a background daemon thread using Python's `threading` module. A `threading.Lock()` protects the shared grid state. The main thread exclusively handles rendering (at 60 FPS) and event polling, while the simulation thread handles search and sleeping between steps. This separation completely eliminated UI freezes.

5.3 Dropdown Z-Order and Click Passthrough

When the algorithm dropdown was open, clicking an option was also registering as a click on the button rendered behind it in screen space, inadvertently triggering Start or Reset.

Solution: The event handler was restructured to give the dropdown first priority when it is open. Any `MOUSEBUTTONDOWN` event that lands within the dropdown's expanded rectangle is consumed and returned early, preventing propagation to widgets below. The dropdown is also drawn last each frame so it visually floats above all other panel elements.

6. Reflection and Conclusion

The dynamic environment fundamentally changes the nature of pathfinding. In a static environment, an algorithm like A* can plan a complete optimal path from start to goal and follow it exactly. Here, that path is invalidated after every single step because the obstacle map changes. The agent must therefore treat each step as an independent planning problem with a fresh map.

This has several interesting consequences. First, the agent's actual traversal (shown in red heat-map shades) diverges significantly from the final green optimal path, which is computed on the fixed obstacle configuration present at the moment the goal is reached. The heat-map often shows cells visited multiple times as the agent was rerouted, while the green path shows what a direct route would look like in a static version of that final map.

Second, the choice of algorithm matters more for speed and visual quality than for guaranteed optimality in the traditional sense. Since no algorithm can guarantee the globally optimal dynamic route (because future obstacle configurations are unknown), the relevant metric becomes efficiency i.e. how close the agent's actual step count is to the static optimal length. A* performed best on this metric across all test cases.

Third, the heat-map visualisation revealed an additional insight: cells near the diagonal between start and goal were revisited far more often than peripheral cells. This makes geometric sense as the heuristic-guided algorithms concentrate exploration along the direct route, and obstacle re-randomisation repeatedly blocks that corridor.

Given more time, several improvements would be worthwhile. Implementing a windowed or incremental version of A* could allow the algorithm to reuse search results from the previous step rather than restarting from scratch, substantially reducing computation per step. Adding a side-by-side comparison mode would allow real-time visual comparison of two algorithms.

Overall, this lab provided valuable hands-on experience with state-space search, dynamic environment design, threading-based animation, and the practical gap between theoretical algorithm properties and real-world dynamic behaviour.

7. References

1. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. (*Chapters 3–4 for uninformed and heuristic search algorithms*)
2. Python Software Foundation. (2024). *Python 3 Documentation: collections.deque, heapq, threading*. <https://docs.python.org/3/>
3. Pygame Community. (2024). *Pygame Documentation*. <https://www.pygame.org/docs/>

Note: The AI tool *Claude (Anthropic)* was consulted in a limited capacity during this assignment, specifically for assistance with boilerplate Pygame widget scaffolding (button rendering, event loop structure) and minor formatting of the written report. All search algorithm implementations (DFS, BFS, UCS, Best-FS, A), the grid logic, the dynamic obstacle system, the threading architecture, and the analytical content of this report are the author's own original work. The AI tool did not contribute to the core algorithmic or problem-solving components of the submission.